



PONTIFICIA UNIVERSIDAD CATOLICA DE CHILE
ESCUELA DE INGENIERIA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACION

Criptografía y Seguridad Computacional - IIC3253
Rúbrica Tarea 3

Preguntas

1. En esta pregunta deberá demostrar la correctitud de la formula de Garner, la cual se utiliza para decriptar mensajes usando RSA en OpenSSH. Recuerde que usamos la notación $a^{-1} \bmod b$ para referirnos al inverso de a en módulo b .

Sean (e, N) y (d, N) un par de llaves RSA pública y privada (respectivamente), con $N = P \cdot Q$.

- (a) [2 puntos] Demuestre que para cada par de números $x_1 \in \{0, \dots, P-1\}$ y $x_2 \in \{0, \dots, Q-1\}$, si existe un número $x \in \{0, \dots, N-1\}$ tal que $x \equiv_P x_1$ y $x \equiv_Q x_2$, dicho número es el único que cumple tal condición. Esto es un caso particular del teorema chino de los restos. No utilice dicho teorema en la demostración.
- (b) [2 puntos] Demuestre que para cada texto cifrado $c \in \{1, \dots, N\}$, su correspondiente texto plano se puede calcular como

$$m = (c^d \cdot P \cdot (P^{-1} \bmod Q) + c^d \cdot Q \cdot (Q^{-1} \bmod P)) \bmod N$$

- (c) [2 puntos] Demuestre que para cada mensaje cifrado $c \in \{1, \dots, N\}$, su correspondiente texto plano se puede calcular como

$$m = c^d \bmod Q + ((c^d \bmod P - c^d \bmod Q) \cdot (Q^{-1} \bmod P)) \bmod P \cdot Q$$

Esta fórmula se conoce como la fórmula de Garner, y es lo que se utiliza en la práctica para firmar y decriptar mensajes usando RSA.

Corrección. La asignación de puntajes para cada pregunta de este problema es la siguiente.

- (a) Tiene 1 punto si la idea de la demostración está correcta pero la formalización está incompleta o con errores menores, y tiene 2 puntos si la demostración está completamente correcta. En esta pregunta no recibe puntaje si utiliza el teorema chino de los restos para realizar la demostración.

- (b) Tiene 1 punto si la idea de la demostración está correcta pero la formalización está incompleta o con errores menores, y tiene 2 puntos si la demostración está completamente correcta. Para resolver esta pregunta se puede utilizar la pregunta (a) aunque no la haya resuelto correctamente, o hacer referencia al teorema chino de los restos.
- (c) Tiene 1 punto si la idea de la demostración está correcta pero la formalización está incompleta o con errores menores, y tiene 2 puntos si la demostración está completamente correcta. Para resolver esta pregunta se puede utilizar la pregunta (a) aunque no la haya resuelto correctamente, o hacer referencia al teorema chino de los restos.
2. En esta pregunta usted deberá mostrar la correctitud del protocolo RSA entre múltiples partes y luego programar una clase que represente a un participante de dicho protocolo. El objetivo de este protocolo es permitir a un grupo de participantes comunicarse de tal forma que todo participante del grupo puede encriptar mensajes que puede decriptar cualquier participante del grupo.

El protocolo RSA entre múltiples partes funciona de la siguiente forma: Supongamos que tenemos ℓ participantes. Cada participante $i \in \{1, \dots, \ell\}$ genera dos primos grandes P_i y Q_i al azar y comparte su *módulo* $N_i = P_i \cdot Q_i$. Luego el grupo verifica que todos los N_i sean primos relativos; de lo contrario el protocolo comienza de nuevo. Finalmente, el grupo se pone de acuerdo en un *exponente público* $e \in \{1, \dots, \min(N_1, \dots, N_\ell)\}$ (se puede sacar al azar) y verifican que sea primo relativo con $\phi(N_i)$ para cada i . Si no lo es, el protocolo elige un nuevo valor $e \in \{1, \dots, \min(N_1, \dots, N_\ell)\}$ y se repite el procedimiento. Definimos la *llave pública* para encriptar como $(e, N_1, N_2, \dots, N_\ell)$.

Si un participante $i \in \{1, \dots, \ell\}$ quiere encriptar un mensaje $m \in \{0, \dots, \min(N_1, \dots, N_\ell) - 1\}$, primero debe calcular, para cada $j \in \{1, \dots, \ell\}$, el valor N_{-j} que es la multiplicación de todos los valores N_k con $k \in \{1, \dots, \ell\} \setminus \{j\}$. Luego, computa $M_j = N_{-j}^{-1} \bmod N_j$. Finalmente, envía el mensaje

$$c = (m^e \bmod N_1) \cdot M_1 \cdot N_{-1} + (m^e \bmod N_2) \cdot M_2 \cdot N_{-2} + \dots + (m^e \bmod N_\ell) \cdot M_\ell \cdot N_{-\ell}$$

- (a) [2 puntos] Explique cómo puede un participante decriptar el mensaje c . Explique cómo se vería su llave secreta y demuestre que el protocolo es correcto (es decir, al decriptar se obtiene el mensaje original). Es importante que su protocolo pueda ser implementado con la interfaz que se describe en la pregunta (b).
- (b) [4 puntos] Escriba una clase en Python que represente a un participante de este protocolo. En particular, su clase deberá respetar al menos la siguiente interfaz.

```
class MultiPartyRSAParticipant:
    def __init__(self, bit_length: int) -> None:
        # Initializes P, Q and N such that N has at least bit_length and at
        # most bit_length + 1 bits

    def get_N(self):
        # Returns N

    def join_group(self, public_exponent: int, group_moduli: List[int]) -
        > boolean:
        # Raises an AssertionError if a group has already been joined.
```

```

        # Verifies that N occurs exactly once in group_moduli and that all
        # other moduli are coprime with N. Otherwise, returns False.
        # Verifies that e is coprime with (P - 1)(Q - 1). Otherwise,
        # returns False.
        # Precomputes and stores all relevant information for encryption
        # and decryption, then returns True.

def enc(self, m: int) -> int:
    # Encrypt m according to the multiparty RSA protocol. If no group
    # has been joined raise an AssertionError.

def dec(self, c: int) -> int:
    # Decrypt c according to the multiparty RSA protocol. If no group
    # has been joined raise an AssertionError.

```

A continuación se muestra un bloque de código que utiliza la clase anterior, ejemplificando el funcionamiento del protocolo:

```

import random

participant_number = 5
bit_length = 2048

participants = []
for i in range(participant_number):
    participants.append(MultiPartyRSAParticipant(bit_length))

moduli = [p.get_N() for p in participants]
public_exponent = random.randint(1, min(moduli))

for p in participants:
    p.join_group(public_exponent, moduli)

encryptor = random.choice(participants)

plaintext = random.randint(0, min(moduli) - 1)
cyphertext = encryptor.enc(plaintext)

for p in participants:
    if p.dec(cyphertext) != message:
        raise AssertionError("Incorrect decryption")

print("All good")

```

Corrección. La asignación de puntajes para cada pregunta de este problema es la siguiente.

- (a) Tiene 1 punto por indicar cómo se calcula la clave secreta de cada participante, y tiene 1 punto por demostrar correctamente que un participante puede descifrar un mensaje cifrado utilizando su clave secreta.
- (b) La corrección de esta pregunta va a estar basado en el siguiente conjunto de tests.
- [0.25 puntos] Al inicializar un participante con un valor `bit_length`, el valor N generado debe ser de la forma $N = P \cdot Q$, donde P y Q son primos, y N debe tener al menos `bit_length` bits y a lo más `bit_length + 1` bits.
 - [0.25 puntos] Si se utiliza `enc` antes de que el participante haya ingresado exitosamente a un grupo mediante `join_group`, se debe obtener una excepción.
 - [0.25 puntos] Si se utiliza `dec` antes de que el participante haya ingresado exitosamente a un grupo mediante `join_group`, se debe obtener una excepción.
 - [0.25 puntos] Si se utiliza `join_group` con una lista `group_moduli` en la que el valor N del participante no aparece, se debe retornar `False`.
 - [0.25 puntos] Si se utiliza `join_group` con una lista `group_moduli` en la que el valor N del participante aparece dos veces, se debe retornar `False`.
 - [0.25 puntos] Si se utiliza `join_group` con una lista `group_moduli` que contiene algún valor distinto de N que no es primo relativo con N , se debe retornar `False`.
 - [0.25 puntos] Si se utiliza `join_group` con un exponente público e que no es primo relativo con $\varphi(N) = (P - 1)(Q - 1)$, se debe retornar `False`.
 - [0.25 puntos] Si un participante ya ingresó exitosamente a un grupo mediante el método `join_group`, un segundo llamado a `join_group` sobre el mismo participante debe producir una excepción.
 - [1 punto] Para un grupo válido de ℓ participantes, si un participante instanciado utilizando su código cifra un mensaje $m \in \{0, \dots, \min(N_1, \dots, N_\ell) - 1\}$ utilizando `enc`, entonces todos los participantes que sigan correctamente el protocolo deben ser capaces de descifrar dicho mensaje.
 - [1 punto] Para un grupo válido de ℓ participantes, si c es el resultado de cifrar un mensaje $m \in \{0, \dots, \min(N_1, \dots, N_\ell) - 1\}$ por un participante instanciado utilizando su código, entonces todos los participantes del grupo deben recuperar exactamente m al utilizar `dec` con entrada c .